

Learning Objects for Introductory Programming:

Foundations, Evaluation, and the Case of Documented Programs

A Combined Research Report

April 2026

This report synthesizes peer-reviewed research on learning objects (LOs) for introductory programming courses, covering their design, evaluation frameworks, empirical evidence of effectiveness, and the specific question of whether a documented program with accompanying materials qualifies as a learning object.

Part I: Learning Objects for Introductory Programming — What Two Decades of Research Reveal

1.1 Introduction

Learning objects (LOs) have shown measurable promise in improving outcomes for introductory programming courses, yet the field remains fragmented and no dedicated systematic review exists on this specific intersection. The strongest evidence comes from the London Metropolitan University studies by Boyle, Bradley, and colleagues, who documented 12–23 percentage point improvements in pass rates after deploying multimedia LOs for Java programming across over 600 students (Bradley & Boyle, 2004). However, the literature also exposes a fundamental tension—Wiley’s *reusability paradox*—where pedagogically rich LOs become harder to reuse across contexts, a problem especially acute in programming education where concepts are deeply interconnected and language-dependent (Wiley, 2002). Research peaked between 2002 and 2012, with a recent resurgence driven by AI-generated learning objects and adaptive learning systems.

1.2 Definitions and Conceptual Frameworks

The concept of “learning objects” draws its name directly from object-oriented programming, coined by Wayne Hodgins in 1994. Despite three decades of use, no consensus definition exists. Churchill (2007) called it “an ill-defined concept, despite numerous and extensive discussion in the literature.” The most commonly cited definitions range from the IEEE LTSC’s broad formulation—“any entity, digital or non-digital, that may be used for learning, education or training” (IEEE, 2002)—to Wiley’s more focused “any digital resource that can be reused to support learning” (Wiley, 2000), to Chiappe, Segovia, and Rincón’s (2007) operational definition requiring three internal components: content, learning activities, and elements of context.

In programming education specifically, researchers have operationalized LOs as small, self-contained multimedia resources covering a single programming concept, typically incorporating text explanations, code examples, interactive animations, and embedded assessments. Boyle (2003) established the most influential design framework for programming LOs, drawing on software engineering principles: each LO should exhibit **cohesion** (addressing a single learning objective) and **de-coupling** (minimal dependencies on other LOs). His “compound learning object” model—combining explanatory text with linked animations and simulations—became the template adopted by most subsequent programming LO projects.

Wiley's (2000) taxonomy classified LOs into five types (fundamental, combined-closed, combined-open, generative-presentation, and generative-instructional), while Churchill's (2007) six-category typology (presentation, practice, simulation, conceptual model, information, and contextual representation) has also been applied in CS education contexts. The core characteristics shared across definitions are discoverability through metadata, reusability across contexts, interoperability across platforms, self-containment, and aggregability into larger instructional sequences.

1.3 Landmark Empirical Studies

The strongest body of empirical evidence comes from two research groups. The **London Metropolitan University group** (Boyle, Bradley, Chalk, Jones, and Pickard) conducted the most cited studies, deploying 54 LOs (40 text-based HTML pages and 14 multimedia Flash objects with animations) for introductory Java programming via WebCT. Their 2003 study in the *Journal of Educational Media* documented pass rate improvements of +19 points for HND, +15 for BSc, and +12 for MSc modules compared to prior years (Boyle et al., 2003). Bradley and Boyle's (2004) follow-up at ED-MEDIA confirmed that 78% of students found animated LOs helpful, with server logs showing extensive voluntary use. The approach won the 2004 European Academic Software Award.

The **University of Nebraska–Lincoln group** (Nugent, Soh, Samal, Miller, and colleagues) produced the most methodologically rigorous work. Their 2006 paper in the *Journal of Educational Technology Systems* validated SCORM-compliant LOs for CS1, confirming viability for independent learning (Nugent et al., 2006). Miller et al.'s (2011) SIGCSE paper analyzed over 1,400 learning sessions across five LOs and found that students with active participation in LO exercises scored significantly higher than those who passively watched.

Other notable empirical contributions include: Villalobos, Calderón, and Jiménez (2009) at Universidad de los Andes with the Cupi2 project; Begosso, Begosso, and Begosso (2015, 2016) at IEEE Frontiers in Education showing that LOs for sorting algorithms increased student motivation while reducing dropout; Tuparov and Tuparova (2012, 2014) demonstrating simulation-based LOs for sorting and searching algorithms; and a controlled experiment at the University of Aguascalientes using Codewitz-adapted LOs for C++ that found no significant difference between groups (Arevalo Mercado, Muñoz Andrade, & Gómez Reynoso, 2008). Most studies are quasi-experimental or case studies; true randomized controlled experiments remain rare.

1.4 Frameworks, Standards, and Design Challenges

Programming LOs sit at the intersection of several technical standards. The SCORM (Sharable Content Object Reference Model) standard provides the dominant packaging and delivery mechanism, integrating IEEE LOM metadata, IMS Content Packaging, and sequencing rules. IEEE LOM (1484.12.1, revised 2020) specifies nine metadata categories enabling discovery and reuse. Several studies explicitly developed SCORM-compliant programming LOs, including Costelloe, Sherry, and Magee (2009) in the *International Journal on E-Learning*, who deployed SCORM-compliant LOs across three Irish institutions.

Boyle's cohesion/de-coupling framework (2003) remains the most influential design approach. He later extended this into Generative Learning Objects (GLOs), which capture successful learning designs—not just content—as reusable templates, supported by the GLO-Maker authoring tool developed at the UK's Centre for Excellence in Reusable Learning Objects (RLO-CETL) (Boyle & Cook, 2004). Leal and Queirós (2009, 2013) developed PEXIL and YAPEXIL—specialized interoperability languages for programming exercises—addressing the unique challenge that programming LOs require code execution, automated evaluation, and feedback capabilities that generic standards cannot natively provide.

1.5 Why Programming LOs Face Unique Challenges

The reusability paradox, articulated by Wiley (2002), is particularly acute for programming content: a Java loop tutorial rich with language-specific syntax, IDE screenshots, and compiler feedback is highly effective but cannot be reused for Python. A language-agnostic explanation of iteration is reusable but pedagogically weak. Beyond this paradox, the literature identifies several domain-specific obstacles: code execution requirements mean programming LOs need compilers or simulation engines that SCORM does not support; concept interdependency makes de-coupling difficult; rapid language evolution causes programming LOs to become obsolete faster than materials in more stable disciplines; and production costs for high-quality multimedia LOs with interactive animations remain prohibitive (Weller, 2020).

1.6 The Systematic Review Gap

No dedicated systematic review or meta-analysis exists on learning objects specifically for introductory programming. The closest works are broader reviews: Luxton-Reilly et al. (2018) produced the most comprehensive systematic literature review of introductory programming at ITiCSE, covering over 700 papers from 2003–2017, but did not isolate LOs as a category. Vihavainen, Airaksinen, and Watson (2014) at ICER meta-analyzed 60 intervention studies, finding teaching interventions can improve pass rates by nearly

one-third. The most recent frontier is AI-generated learning objects: Lohr et al. (2025) in the *Journal of Computer Assisted Learning* used GPT-4-turbo with retrieval-augmented generation to produce semantically annotated LOs for CS courses, finding that generated question quality often did not meet educational standards and required significant human intervention.

Part II: Evaluating Learning Objects for Introductory Programming

2.1 Introduction

No validated evaluation instrument exists specifically for programming learning objects, despite two decades of research on LO quality assessment. The field relies on general-purpose frameworks—primarily LORI, LOES-S, and MERLOT’s peer review criteria—none of which adequately address code execution, debugging feedback, or language-version obsolescence unique to programming content. A 2007 review of 58 LO evaluation studies found only two that measured actual learning gains (Kay & Knaack, 2007).

2.2 Three Dominant Evaluation Frameworks

2.2.1 The Learning Object Review Instrument (LORI)

The LORI, developed by Nesbit, Belfer, and Leacock (2007) at Simon Fraser University, is the most widely cited general-purpose LO evaluation tool. LORI 1.5 rates objects on nine dimensions using a 1–5 scale: content quality, learning goal alignment, feedback and adaptation, motivation, presentation design, interaction usability, accessibility, reusability, and standards compliance. Its theoretical grounding draws on Mayer’s (2001) cognitive theory of multimedia learning and Sweller’s (1988) cognitive load theory. LORI operates within a convergent participation model—multiple reviewers rate independently, then engage in structured discussion. However, Nurmi and Jaakkola (2006) showed that LORI ratings did not correlate with actual learning gains in a K–12 validation study.

2.2.2 The Learning Object Evaluation Scale for Students (LOES-S)

Kay and Knaack’s (2009) LOES-S, published in *Educational Technology Research and Development*, measures three constructs—learning, quality/instructional design, and engagement—through 12 Likert items plus two open-ended questions. Validated with over 1,100 students, LOES-S demonstrated strong psychometric properties: Cronbach’s alpha of 0.89 for learning, 0.84 for quality, and 0.78 for engagement, with confirmatory principal component analysis and significant predictive validity. Their companion Learning Object Evaluation Metric (LOEM), published in the *Australasian Journal of Educational Technology* (Kay & Knaack, 2008), targets teachers using four constructs: interactivity, design, engagement, and usability.

2.2.3 MERLOT Peer Review

MERLOT's peer review process evaluates materials on three dimensions: quality of content, potential effectiveness as a teaching tool, and ease of use. Two peer reviewers independently evaluate each resource on a 1–5 scale, and materials scoring 3 or above are published. MERLOT now hosts over 107,000 resources, though the labor-intensive process means only a fraction receives formal peer review (Hilera et al., 2020).

2.3 Programming LOs Lack Purpose-Built Evaluation Criteria

None of the dominant frameworks were designed with executable, interactive programming content in mind. Several programming-relevant dimensions fall entirely outside existing instruments: code correctness and style quality; progressive scaffolding of problem-solving, aligning with van Merriënboer and De Croock's (1992) evidence on program completion strategies; language-version sustainability; and integration with development environments. Börstler et al. (2008) developed a CS-education-specific instrument for evaluating OO example programs at ITiCSE, but it targets static code rather than interactive LOs. The CT-PAF framework (de Computación Creativa et al., 2022) evaluates computational thinking tools but focuses on visual programming environments like Scratch.

Cognitive load theory provides the strongest theoretical lens for programming LO evaluation. A 2025 study in *ACM Transactions on Computing Education* confirmed that worked examples with in-code commenting reduced extraneous load and enhanced both retention and transfer for novice programmers (Kim & Ko, 2025). Berssanette and de Francisco's (2022) systematic review in *IEEE Transactions on Education* established that effective programming LOs must minimize extraneous load through integrated code-and-explanation presentation.

2.4 Empirical Evidence on Programming LO Effectiveness

Kay and Knaack's (2007) landmark review in *Open Learning* examined 58 published articles on LO evaluation and identified six pervasive problems: emphasis on technology over learning, absence of reliability and validity estimates, over-reliance on informal descriptive data, tendency toward general impressions rather than specific design features, use of either formative or summative evaluation but rarely both, and testing on small or vaguely described samples.

The most substantial programming-specific evaluation remains Bradley and Boyle's (2004) work at London Metropolitan University. Their mixed-methods evaluation combined four questionnaires, student interviews, and WebCT tracking data across over 600 students. Student satisfaction reached 80–90%, with animation LOs rated

“very useful” by 46%—the highest rating among all course resources including lectures (37%) and textbooks (18%). The only controlled experiment (Arevalo Mercado et al., 2008) yielded null results with approximately 36 students, acknowledging limitations of single-lesson scope and small sample size.

2.5 Measuring Learning Outcomes in CS1

Researchers measure programming LO effectiveness through four primary channels. **Pass rates** remain the most common metric: Bennedsen and Caspersen (2007) established the worldwide CS1 baseline at approximately 67% from 63 institutions; Watson and Li (2014) confirmed this at 67.7% across 161 courses in 15 countries. **Concept inventories** represent the gold standard: the FCS1 by Tew and Guzdial (2011) was the first language-independent validated assessment; Parker et al. (2016) produced the isomorphic SCS1 replication. **Self-efficacy measurement** provides a complementary dimension: Ramalingam and Wiedenbeck’s (1998) CPSES remained the standard until Steinhorst, Petersen, and Vahrenhold’s (2020) language-independent IPSES. **Automated assessment** systems are increasingly applied: Keuning, Jeuring, and Heeren’s (2019) systematic review in *ACM TOCE* analyzed 101 tools, and Messer et al.’s (2024) follow-up covered 121 papers.

2.6 Repository Quality Assurance

Beyond MERLOT, quality processes vary dramatically. OER Commons employs the Achieve OER Rubrics across eight dimensions. Jorum, the UK’s JISC-funded national repository (now retired), applied IEEE LOM metadata standards. Tzikopoulos, Manouselis, and Vuorikari’s (2009) survey of 59 well-known LO repositories found that only 46% followed any quality control policy and just 39% had a resource assessment or rating policy. Clements, Pawlowski, and Manouselis (2015) in *Computers in Human Behavior* conducted the first systematic review on repository quality approaches, finding that user-generated collaborative approaches were favored for sustainability but lack the rigor of expert review.

2.7 Critical Gaps in Programming LO Evaluation

Seven critical gaps define the current state: (1) no validated instrument exists specifically for evaluating programming LOs; (2) no systematic review or meta-analysis focuses on LO evaluation in CS education; (3) rigorous controlled experiments are nearly absent; (4) validated CS1 concept inventories (FCS1, SCS1) are available but rarely used in LO evaluation studies; (5) sustainability and obsolescence criteria are missing from every framework; (6) learning analytics methods are mature enough but remain disconnected from LO evaluation; and (7) automated quality assessment

leveraging static analysis and code correctness checking represents an unexplored intersection.

Part III: A Documented Program as a Learning Object

3.1 Introduction

A question that naturally arises from the preceding analysis is whether a single, well-documented small program—accompanied by a written report and a video walkthrough of the code—can legitimately be classified as a learning object. This part examines the bundle against five major LO definitions, maps it onto established taxonomies, and reviews precedent from both the literature and actual learning object repositories.

3.2 Testing Against Five Major Definitions

The **IEEE LTSC 1484.12.1** (2002) definition—“any entity, digital or non-digital, that may be used for learning, education, or training”—is the broadest, and the bundle qualifies trivially. **Wiley’s (2000)** definition requires a digital resource that can be reused to support learning, imposing four attributes: digital, resource, supports learning, and reusable. A documented program with report and video walkthrough satisfies all four, though Wiley’s Reusability Paradox applies—the more context-specific the bundle, the less reusable it becomes.

Polsani (2003) requires “an independent and self-standing unit of learning content that is predisposed to reuse in multiple instructional contexts,” grounded in accessibility, reusability, and interoperability. The bundle meets the independence and self-standing criteria strongly: the report provides the framework embedding the digital object, while the video provides the exposition facilitating understanding. The key caveat is that Polsani insists the LO should avoid references to specific institutional contexts.

Churchill (2007) defines a learning object as “a representation designed to afford uses in different educational contexts,” and the bundle qualifies provided it was designed for broader applicability.

Chiappe, Segovia, and Rincón (2007) impose the most demanding criteria, requiring content, learning activities, and context elements plus external metadata. The source code and report supply content; the report can supply context elements. However, if the bundle consists only of passive materials—a video walkthrough without embedded exercises or reflection prompts—the learning activities requirement is not fully met. Adding explicit exercises or guided tasks would resolve this gap.

3.3 Taxonomic Classification

Under **Wiley’s five-type taxonomy**, the bundle is a *combined-open* learning object. Its constituent resources (source code, PDF report, video file) remain individually recoverable and reusable—a learner can extract the source code for a different

assignment, repurpose the report as a reading, or show the video independently. This distinguishes it from a combined-closed object, where components are fused and irrecoverable (Wiley, 2000).

Under **Churchill's six-category typology**, the bundle is primarily a *presentation object*—its dominant function is to transmit and explain subject matter. However, the source code introduces latent practice object affordances: learners can compile, execute, modify, and experiment with the code (Churchill, 2007). Under the **IEEE LOM aggregation levels**, the bundle sits at Level 2—a collection of Level 1 raw media fragments assembled into a lesson-sized unit. Under **SCORM's content aggregation model**, the three components are each assets, and the bundle maps to a single SCO (Sharable Content Object).

3.4 Boyle's Framework: An Ideal Compound Learning Object

Boyle's (2003) cohesion/de-coupling framework provides the most illuminating lens. **Cohesion is strong**: all three components serve a single purpose—understanding one specific program. The code is the artifact, the report explains the design decisions, and the video walks through execution. **De-coupling is achievable**: a self-contained program bundle with no references to external course materials can be lifted from one course and dropped into another without modification.

The bundle is structurally identical to Boyle's own compound learning objects for Java programming at London Metropolitan University, which consisted of core text explanations with code examples linked to multimedia expansion objects—Flash animations that displayed, explained, and animated code execution (Boyle et al., 2003; Bradley & Boyle, 2004). The program-report-video bundle maps precisely onto this architecture: the report is the core text explanation, the source code is the executable component, and the video walkthrough is the multimedia expansion. This architecture produced the 12–19 percentage point pass-rate improvements documented in Part I.

3.5 Precedent in CS Education Literature

Adamchik and Gunawardena (2003) at Carnegie Mellon University created a repository of learning objects for programming education where each LO consisted of core material, code examples, supplementary notes, and review questions. Brusilovsky et al. (2014) at the University of Pittsburgh developed PCEX (Program Construction Examples)—interactive line-by-line code walkthroughs treated as smart learning objects within the SPLICE ecosystem. The most robust formal precedent is **crimsonHex** (Leal & Queirós, 2009, 2013), a purpose-built repository that formally defines programming problems as learning objects with extended IEEE LOM metadata, connected to the EU-funded EduJudge project serving over 100,000 users.

3.6 Repository Precedents

MERLOT's Computer Science category contains over 11,652 catalogued resources, with the Programming subcategory containing over 4,671 items, including tools like Python Tutor and documented project collections. OER Commons provides the closest direct precedent: a catalogued lesson combining video walkthroughs with source code for students to study. The Carpentries (formerly Software Carpentry) operate at scale on GitHub with openly licensed lesson materials, each containing code examples, exercises, and instructor guides—and a 2015 blog post explicitly discussed their lessons in the context of learning object theory (Wilson, 2015). Minguillón (2010) at the Universitat Oberta de Catalunya explicitly listed source code files (C, Java) as a recognized type of learning object.

3.7 Arguments For and Against

The case **for** treating the bundle as a learning object rests on six pillars: self-containment, appropriate granularity (the “lesson-sized” sweet spot), metadata taggability via IEEE LOM fields, aggregability into course sequences, strong cohesion under Boyle's framework, and extensive precedent in both literature and repositories.

The case **against** centers on three concerns. The Reusability Paradox (Wiley, 2002) means a richly documented program is valuable precisely because of its specificity, which limits reuse. Chiappe et al.'s (2007) learning activity requirement challenges bundles offering only passive consumption. And video editability is limited—once recorded, a walkthrough cannot be easily updated if the code changes, creating a maintenance burden.

3.8 Conclusion: A Textbook Example

The theoretical and practical evidence converges on a clear position: a small, well-documented program with report and video walkthrough is a compound learning object—specifically, a combined-open object at IEEE LOM aggregation Level 2, functioning primarily as a presentation object with practice affordances. The only design recommendation is practical: to satisfy the most restrictive definition (Chiappe et al., 2007) and maximize pedagogical value, the bundle should include explicit learning activities rather than relying solely on passive video viewing. The bundle should also avoid context-specific references, use standard file formats, and be tagged with formal metadata to enable discovery and reuse.

Conclusion

This combined report has examined learning objects for introductory programming from three complementary angles. Part I surveyed two decades of research on LO design and deployment in CS1 courses, finding that well-designed, interactive LOs can produce meaningful pass-rate improvements (12–23 percentage points in the strongest study) but that the field suffers from a lack of controlled experiments and the persistent reusability paradox. Part II revealed that no validated evaluation instrument exists specifically for programming LOs, with the field relying on general-purpose frameworks (LORI, LOES-S, MERLOT) that do not address code execution, debugging feedback, or language evolution. Part III demonstrated that a documented program with report and video walkthrough clearly qualifies as a compound learning object under established definitions and taxonomies, with extensive precedent in both the literature and actual repositories.

Three priorities emerge for researchers entering this space: (1) conducting the first systematic review of LOs specifically in programming education; (2) developing an evaluation framework that addresses the interactive, executable nature of programming content; and (3) investigating how generative AI can address the reusability paradox by enabling rapid contextualization of modular learning materials. The field is ripe for synthesis, and the growing interest in open educational resources and AI-assisted content generation makes the timing particularly opportune.

References

- Adamchik, V., & Gunawardena, A. (2003). A learning objects approach to teaching programming. In *Proceedings of the International Conference on Information Technology: Coding and Computing (ITCC 2003)* (pp. 96–100). IEEE.
- Alammary, A. (2019). Blended learning models for introductory programming courses: A systematic review. *PLOS ONE*, 14(9), e0221765.
- Arevalo Mercado, C. A., Muñoz Andrade, F., & Gómez Reynoso, J. M. (2008). The effect of learning objects on a C++ programming course. *Communications of the IIMA*, 8(4), 63–74.
- Begosso, L. R., Begosso, L. C., & Begosso, R. H. (2015). The use of learning objects for teaching computer programming. In *Proceedings of the IEEE Frontiers in Education Conference (FIE 2015)* (pp. 1–8). IEEE.
- Begosso, L. R., Begosso, L. C., & Begosso, R. H. (2016). Learning objects for teaching computer programming. In *Proceedings of the IEEE Frontiers in Education Conference (FIE 2016)* (pp. 1–5). IEEE.
- Bennedsen, J., & Caspersen, M. E. (2007). Failure rates in introductory programming. *ACM SIGCSE Bulletin*, 39(2), 32–36.
- Berssanette, J. H., & de Francisco, A. C. (2022). Cognitive load theory in the context of teaching and learning computer programming: A systematic literature review. *IEEE Transactions on Education*, 65(3), 440–449.
- Blikstein, P., Worsley, M., Piech, C., Sahami, M., Cooper, S., & Koller, D. (2014). Programming pluralism: Using learning analytics to detect patterns in the learning of computer programming. *Journal of the Learning Sciences*, 23(4), 561–599.
- Börstler, J., Johansson, T., Nordström, M., Paterson, J., Ratcliffe, M., & Schulte, C. (2008). An evaluation of object oriented example programs in introductory programming textbooks. In *Proceedings of the 13th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '08)* (pp. 126–130). ACM.
- Boyle, T. (2003). Design principles for authoring dynamic, reusable learning objects. *Australasian Journal of Educational Technology*, 19(1), 46–58.
- Boyle, T., Bradley, C., Chalk, P., Jones, R., & Pickard, P. (2003). Using blended learning to improve student success rates in learning to program. *Journal of Educational Media*, 28(2–3), 165–178.
- Boyle, T., & Cook, J. (2004). Towards a pedagogically sound basis for learning object portability and re-use. In G. Richards (Ed.), *Proceedings of E-Learn 2004* (pp. 2267–2271). AACE.
- Bradley, C., & Boyle, T. (2004). The design, development, and use of multimedia learning objects. *Journal of Educational Multimedia and Hypermedia*, 13(4), 371–389.
- Brusilovsky, P., Edwards, S., Kumar, A., Malmi, L., Benotti, L., Buck, D., ... & Sosnovsky, S. (2014). Increasing adoption of smart learning content for computer science education. In *Proceedings of the Working Group Reports of the 2014 Innovation and Technology in Computer Science Education Conference (ITiCSE-WGR '14)* (pp. 31–57). ACM.

- Chiappe, A., Segovia, Y., & Rincón, H. Y. (2007). Toward an instructional design model based on learning objects. *Educational Technology Research and Development*, 55(6), 671–681.
- Churchill, D. (2007). Towards a useful classification of learning objects. *Educational Technology Research and Development*, 55(5), 479–497.
- Clements, K. I., Pawlowski, J. M., & Manouselis, N. (2015). Open educational resources repositories literature review: Towards a comprehensive quality approaches framework. *Computers in Human Behavior*, 51, 1098–1106.
- Costelloe, E., Sherry, E., & Magee, P. (2009). Experiences gained using a set of SCORM compliant reusable learning objects for teaching programming. *International Journal on E-Learning*, 8(2), 209–224.
- de Computación Creativa, G., et al. (2022). Developing a pedagogical evaluation framework for computational thinking supporting technologies and tools. *Frontiers in Education*, 7, 957739.
- Gordillo, A., López-Fernández, D., & Verbert, K. (2020). Examining the usefulness of quality scores for generating learning object recommendations in repositories of open educational resources. *Applied Sciences*, 10(13), 4638.
- Hilera, J. R., Hernández-Rizzardini, R., Pagani, R., & Priego, E. (2020). MERLOT peer review process and its impact on educational quality. In *Proceedings of the TEEM Conference* (pp. 1–8). ACM.
- IEEE. (2002). IEEE Standard for Learning Object Metadata (IEEE 1484.12.1-2002). IEEE Standards Association.
- Kay, R. H., & Knaack, L. (2007). Evaluating the learning in learning objects. *Open Learning*, 22(1), 5–28.
- Kay, R. H., & Knaack, L. (2008). A multi-component model for assessing learning objects: The Learning Object Evaluation Metric (LOEM). *Australasian Journal of Educational Technology*, 24(5), 574–591.
- Kay, R. H., & Knaack, L. (2009). Assessing learning, quality and engagement in learning objects: The Learning Object Evaluation Scale for Students (LOES-S). *Educational Technology Research and Development*, 57(2), 147–168.
- Keuning, H., Jeuring, J., & Heeren, B. (2019). A systematic literature review of automated feedback generation for programming exercises. *ACM Transactions on Computing Education*, 19(1), Article 3.
- Kim, J., & Ko, A. J. (2025). Effects of worked examples with explanation types and learner motivation on cognitive load and programming problem-solving performance. *ACM Transactions on Computing Education*, 25(1), Article 12.
- Leal, J. P., & Queirós, R. (2009). CrimsonHex: A repository for programming exercises. In *Proceedings of the International Conference on Computer Supported Education (CSEDU 2009)* (pp. 216–223). INSTICC.
- Leal, J. P., & Queirós, R. (2013). Design of a repository of programming problems. In *Proceedings of the 8th European Conference on Technology Enhanced Learning (EC-TEL 2013)* (pp. 475–480). Springer.

- Lohr, D., Berges, M., Chugh, T., Kohlhase, A., & Müller, W. (2025). Enhancing computing education through large language models. *Journal of Computer Assisted Learning*, 41(1), e70048.
- Luxton-Reilly, A., Simon, Albluwi, I., Becker, B. A., Giannakos, M., Kumar, A. N., ... & Szabo, C. (2018). Introductory programming: A systematic literature review. In *Proceedings of the 23rd Annual ACM Conference on Innovation and Technology in Computer Science Education (ITiCSE '18, Companion)* (pp. 55–106). ACM.
- Mayer, R. E. (2001). *Multimedia Learning*. Cambridge University Press.
- Messer, M., Brown, N. C. C., Kölling, M., & Sherrel, M. (2024). Automated grading and feedback tools for programming education: A systematic review. *ACM Transactions on Computing Education*, 24(1), Article 8.
- Miller, C. S., Soh, L.-K., Samal, A., & Nugent, G. (2011). A comparative analysis of the effects of use and revision of computer science learning objects. In *Proceedings of the 42nd ACM Technical Symposium on Computer Science Education (SIGCSE '11)* (pp. 291–296). ACM.
- Minguillón, J. (2010). Types of learning objects [Presentation slides]. EdReNe Strategic Seminar, Universitat Oberta de Catalunya.
- Nesbit, J. C., Belfer, K., & Leacock, T. L. (2007). A framework for evaluating the quality of multimedia learning resources. *Educational Technology Research and Development*, 55(5), 507–531.
- Nugent, G., Soh, L.-K., & Samal, A. (2006). Design, development, and validation of learning objects. *Journal of Educational Technology Systems*, 34(3), 271–281.
- Nurmi, S., & Jaakkola, T. (2006). Effectiveness of learning objects in various instructional settings. *Learning, Media and Technology*, 31(3), 233–247.
- Parker, M. C., Guzdial, M., & Engleman, S. (2016). Replication, validation, and use of a language independent CS1 knowledge assessment. In *Proceedings of the 2016 ACM Conference on International Computing Education Research (ICER '16)* (pp. 93–101). ACM.
- Polsani, P. R. (2003). Use and abuse of reusable learning objects. *Journal of Digital Information*, 3(4).
- Ramalingam, V., & Wiedenbeck, S. (1998). Development and validation of scores on a computer programming self-efficacy scale and group analyses of novice programmer self-efficacy. *Journal of Educational Computing Research*, 19(4), 367–381.
- Steinhorst, M., Petersen, J., & Vahrenhold, J. (2020). Revisiting self-efficacy in introductory programming. In *Proceedings of the 2020 ACM Conference on International Computing Education Research (ICER '20)* (pp. 158–169). ACM.
- Sweller, J. (1988). Cognitive load during problem solving: Effects on learning. *Cognitive Science*, 12(2), 257–285.
- Tew, A. E., & Guzdial, M. (2011). The FCS1: A language independent assessment of CS1 knowledge. In *Proceedings of the 42nd ACM Technical Symposium on Computer Science Education (SIGCSE '11)* (pp. 111–116). ACM.

- Tuparov, G., & Tuparova, D. (2014). Teaching sorting and searching algorithms through simulation-based learning objects in an introductory programming course. *Procedia – Social and Behavioral Sciences*, 116, 2962–2966.
- Tzikopoulos, A., Manouselis, N., & Vuorikari, R. (2009). An overview of learning object repositories. In T. Halpin (Ed.), *Selected Readings on Database Technologies and Applications* (pp. 29–55). IGI Global.
- van Merriënboer, J. J. G., & De Croock, M. B. M. (1992). Strategies for computer-based programming instruction: Program completion vs. program generation. *Journal of Educational Computing Research*, 8(3), 365–394.
- Vihavainen, A., Airaksinen, J., & Watson, C. (2014). A systematic review of approaches for teaching introductory programming and their influence on success. In *Proceedings of the 10th Annual ACM Conference on International Computing Education Research (ICER '14)* (pp. 19–26). ACM.
- Villalobos, J., Calderón, N., & Jiménez, C. (2009). Developing programming skills by using interactive learning objects. In *Proceedings of the 14th Annual Conference on Innovation and Technology in Computer Science Education (ITiCSE '09)* (pp. 151–155). ACM.
- Watson, C., & Li, F. W. B. (2014). Failure rates in introductory programming revisited. In *Proceedings of the 2014 Conference on Innovation and Technology in Computer Science Education (ITiCSE '14)* (pp. 39–44). ACM.
- Weller, M. (2020). *25 years of ed tech*. Athabasca University Press.
- Wiley, D. A. (2000). Connecting learning objects to instructional design theory: A definition, a metaphor, and a taxonomy. In D. A. Wiley (Ed.), *The Instructional Use of Learning Objects*. Agency for Instructional Technology.
- Wiley, D. A. (2002). The reusability paradox. Retrieved from <https://opencontent.org/blog/archives/3854>
- Wilson, G. (2015, April 8). The paradox of learning objects [Blog post]. The Carpentries. <https://carpentries.org/blog/2015/04/the-paradox-of-learning-objects/>
- Yuan, M., & Recker, M. (2015). A review of rubrics for evaluating the quality of open educational resources. *International Review of Research in Open and Distributed Learning*, 16(3), 18–38.